

Podstawy Sztucznej Inteligencji (PSI)

Lab 2 - omówienie

Pytania? Wątpliwości? Uwagi?

Wstęp do sieci neuronowych

Idea

- zdecydowana większość ma **budowę warstwową**:
 - **neurony (neurons)** układane w **warstwy (layers)**
 - neurony w warstwie nie są ze sobą połączone, tylko warstwy po kolei
 - **wyjście** poprzedniej warstwy to **wejście** następnej
- na wyjściu całej sieci dostajemy np. wyniki dla poszczególnych klas
- tzw. sieci **feedforward**

Rodzaje sieci neuronowych

Rodzaj sieci	Numer laboratorium	Dziedzina zastosowań
Multilayer Perceptron (MLP)	3	Dane tabelaryczne
Convolutional Neural Network (CNN)	4	Obrazy, audio, klasyfikacja szeregów czasowych
Recurrent Neural Network (RNN)	-	Szeregi czasowe, dane sekwencyjne
Transformers	5	Przetwarzanie języka naturalnego (NLP) oraz ostatnio wszystko inne
Graph Neural Networks (GNN)	6 (zad. dodatkowe)	Grafy, dane przestrzenne, ostatnio też obrazy, szeregi czasowe

Inne sieci neuronowe

- autoencoders (AEs)
- generative adversarial networks (GANs)
- diffusion models
- State Space Models (SSMs), e.g. Mamba
- spiking networks

Sieci typu Multilayer Perceptron (MLP)

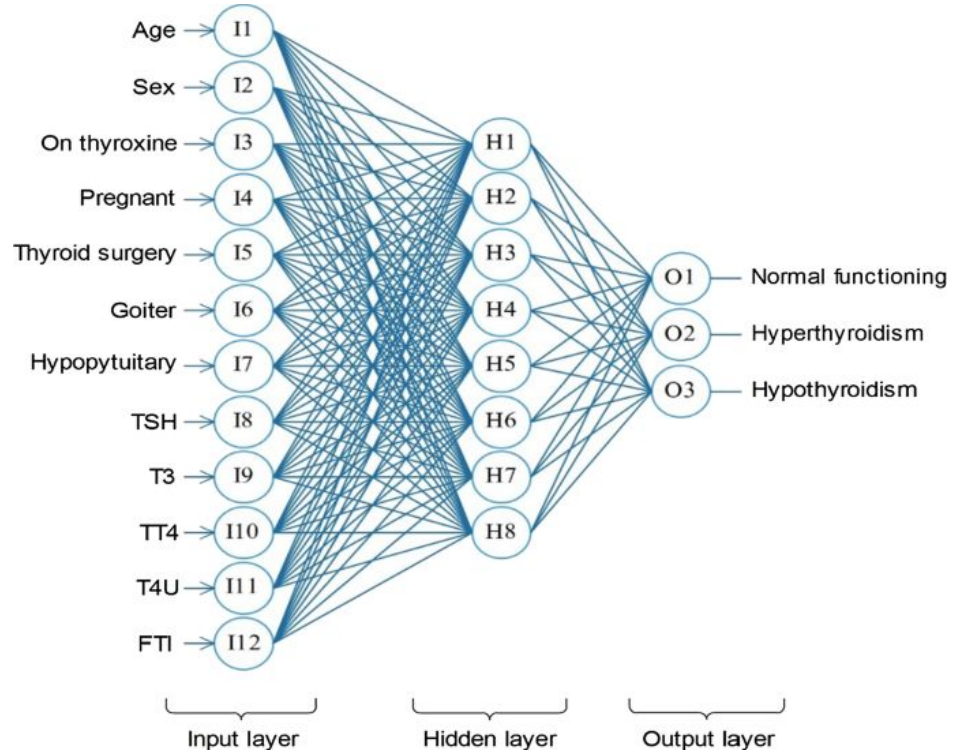
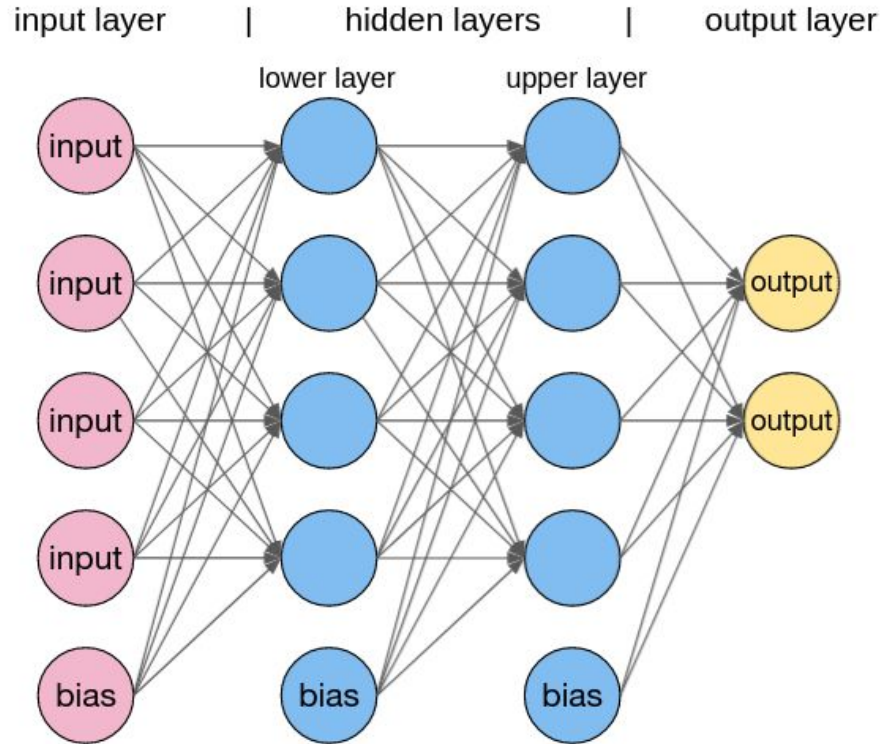
MultiLayer Perceptron (MLP)

- najstarszy i podstawowy rodzaj sieci
- regresja lub klasyfikacja na danych tabelarycznych
- niezbędny element budowy innych sieci, typowo na końcu, jako tzw. **głowa (head)**
- wejście: wektor
- wyjście: skalar (regresja) lub wektor **logitów (logits)**, długość = liczba klas

Budowa MLP

- wejście sieci: wektor
- składa się z **warstw w pełni połączonych (fully / densely connected)**
- wyjście poprzedniej warstwy to wejście następnej
- każdy neuron z poprzedniej warstwy łączy się z każdym z następnej (stąd nazwa)
- wyjście sieci:
 - skalar (regresja)
 - wektor **logitów (logits)**, długość = liczba klas

Budowa MLP

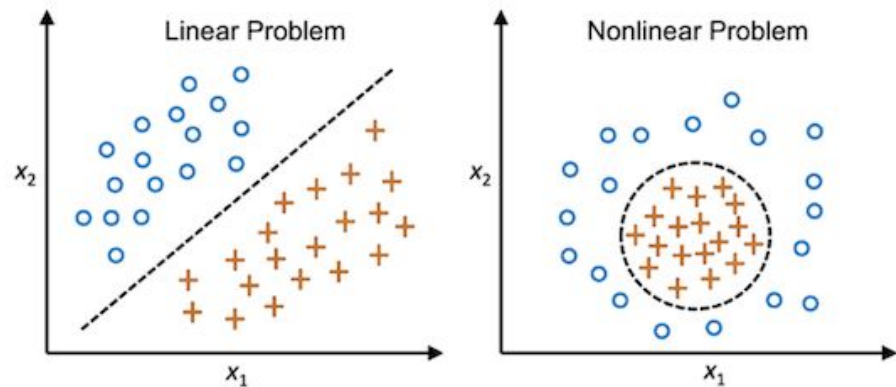


Już to znamy!

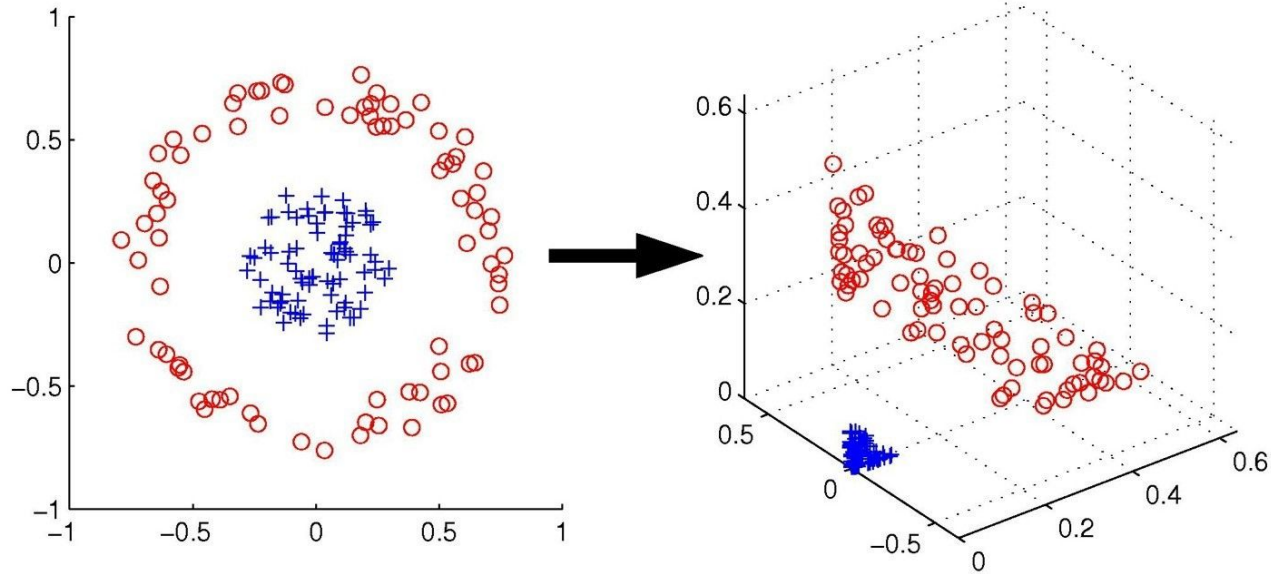
- sieć z jedną warstwą: $f(X) = \sigma(XW^T)$
- jeżeli σ to identyczność (aktywacja liniowa), to mamy **regresję liniową**
- jeżeli σ to sigmoida, to mamy **regresję logistyczną**

Co to tak naprawdę robi?

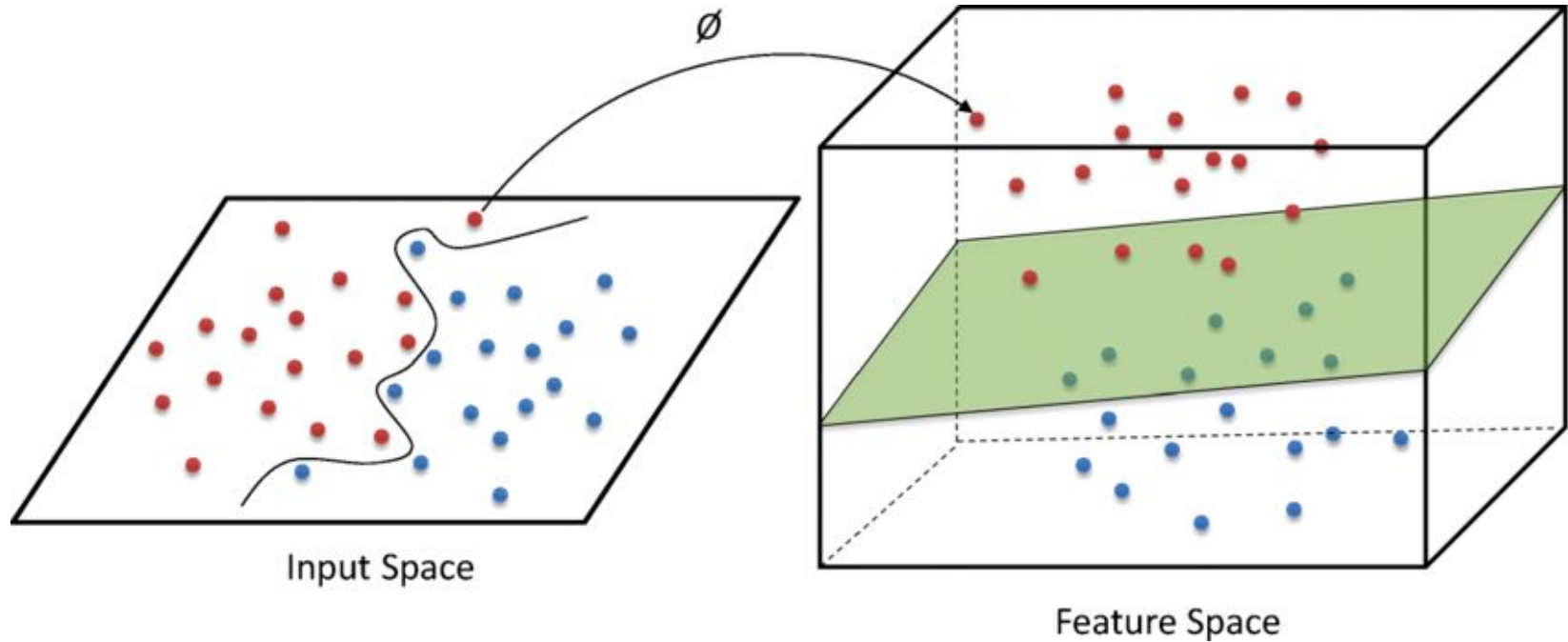
- w ostatniej warstwie (regresja logistyczna) robimy klasyfikację
- zakładamy w niej, że klasy są **liniowo separowalne (linearly separable)**
- poprzednie warstwy to tak naprawdę **transformatory**, które robią **nieliniowe przekształcenie** przestrzeni
- ułatwia to separację klas



Nieliniowe przekształcenia przestrzeni



Nieliniowe przekształcenia przestrzeni



Warstwa w pełni połączona

- zbiór neuronów, każdy to **wektor parametrów**, o zadanej **wymiarowości**
- cała warstwa to **macierz wag W**
- neurony realizują **funkcję**, której wejściem jest macierz X : $f(X) = \sigma(XW^T)$
- σ to **funkcja aktywacji (activation function)**, która wyznacza, jak bardzo wejście **aktywuje** dany neuron
- dodaje się też bias (kolumna 1)
- wyjście nazywa się często **ukrytą reprezentacją (hidden representation)**

Sieć MLP

- mamy warstwy po kolei, przekazują sobie wejście - to **złożenie funkcji**:

$$H_1 = f_1(X) = \sigma(XW_1^T)$$

$$H_2 = f_2(H_1) = \sigma(H_1W_2^T)$$

...

$$H_n = f_{n-1}(H_{n-1}) = \sigma(H_{n-1}W_n^T)$$

- funkcja σ musi być **nieliniowa (non-linear)**, bo złożenie funkcji liniowych byłoby liniowe
- warstwy w środku nazywa się **warstwami ukrytymi (hidden layers)**

Wyjście sieci

Regresja:

- 1 neuron
- "bez aktywacji" (liniowa)

Klasyfikacja:

- K neuronów dla K klas
- wyjście przed aktywacją to **logity (logits)**, wartości rzeczywiste dla każdej klasy
- aktywacja **funkcją softmax** - zamienia logity na rozkład prawdopodobieństw

Trening sieci neuronowych

Trening sieci neuronowych

- **przejsście w przód (forward pass)** - wejście przechodzi przez sieć i dostajemy predykcję
- **funkcja kosztu (loss / cost function)** - oblicza różnicę między predykcją a prawdą
- **cel:** chcemy wiedzieć, jak zmienić poszczególne wagi (parametry sieci), aby uzyskać mniejszy koszt
- aktualizację tę robi **propagacja wsteczna (backward pass)**

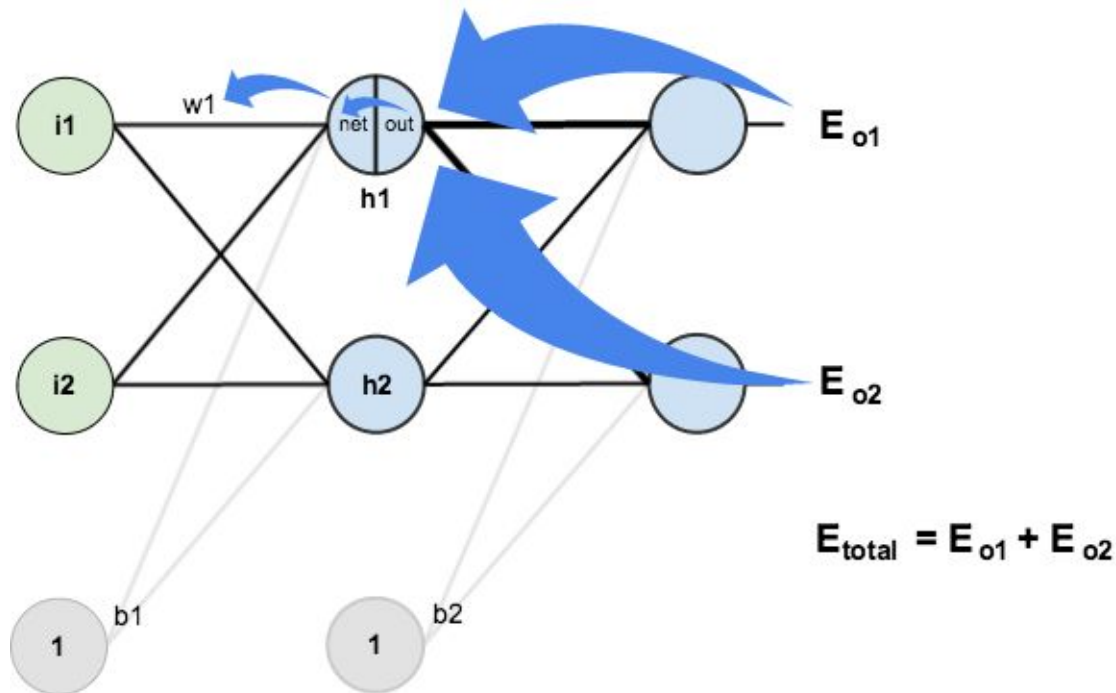
Propagacja wsteczna

- przejście w tył (backward pass)
- liczymy **po pochodną funkcji kosztu** i jej wartość
- za pomocą reguły łańcuchowej liczymy pochodne cząstkowe - pochodne po neuronach z **ostatniej warstwy**
- aktualizujemy zgodnie ze spadkiem wzdłuż gradientu: $w_i = w_i - \alpha \frac{\partial C(W)}{\partial w_i}$
- cofamy się do poprzedniej warstwy, powtarzamy

Propagacja wsteczna

$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial out_{h1}} * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

$$\downarrow$$
$$\frac{\partial E_{total}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial out_{h1}} + \frac{\partial E_{o2}}{\partial out_{h1}}$$



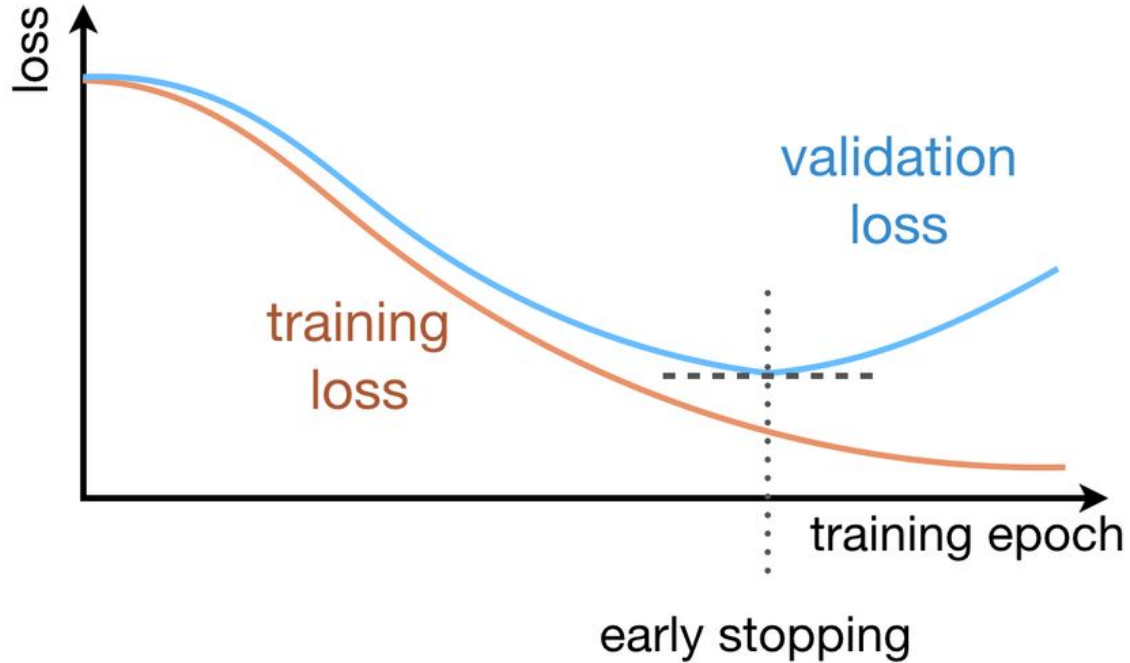
Przybliżanie pochodnych

- używamy tutaj **empirycznego** oszacowania, z danych treningowych
- można to robić:
 - **full-batch GD** - całe dane naraz, nie starczy zwykle pamięci
 - **stochastic GD** - 1 przykład naraz, zwykle słabo używa GPU
 - **mini-batch GD** - paczka przykładów, złoty środek
- mniejszy batch **implicite regularyzuje** trening sieci

Czas treningu

- **epoka (epoch)** - przejście 1 raz przez cały zbiór treningowy
- im dłuższy trening, tym większe ryzyko overfittingu
- **wczesny stop (early stopping)** - przerwanie treningu wcześniej
 - co epokę / ileś mini-batchy sprawdzamy wynik walidacyjny
 - sprawdzamy, czy przez ostatnie X sprawdzeń dostaliśmy nowy najlepszy wynik
 - jeżeli nie, to przerywamy
- X to **cierpliwość (patience)**

Training-validation loss curve

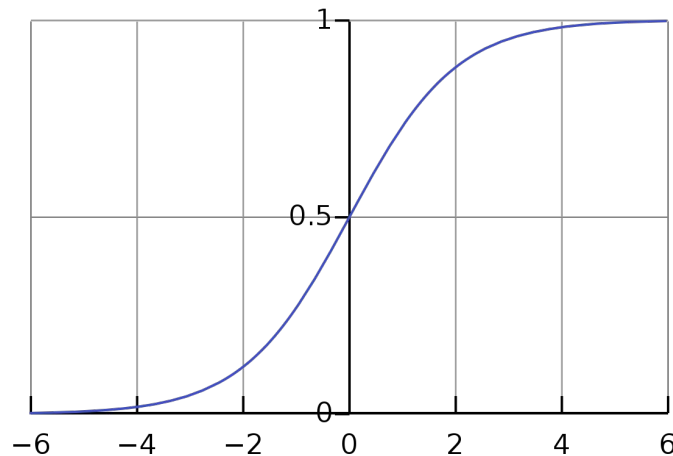


Funkcje aktywacji

Sigmoida

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

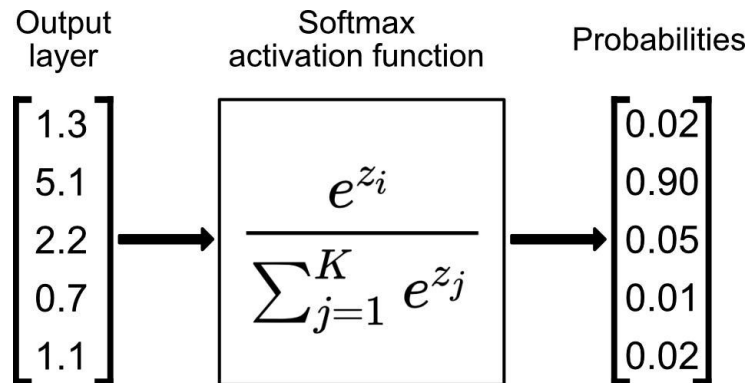
- nieużywane wewnątrz sieci, bo ma płaskie końce
- płaskie = gradienty bliskie 0 = sieć przestaje się uczyć
- problem **nasyconych neuronów (saturated neurons)**
- czasem używane na końcu sieci w klasyfikacji binarnej



Softmax

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{i=1}^K e^{x_i}}$$

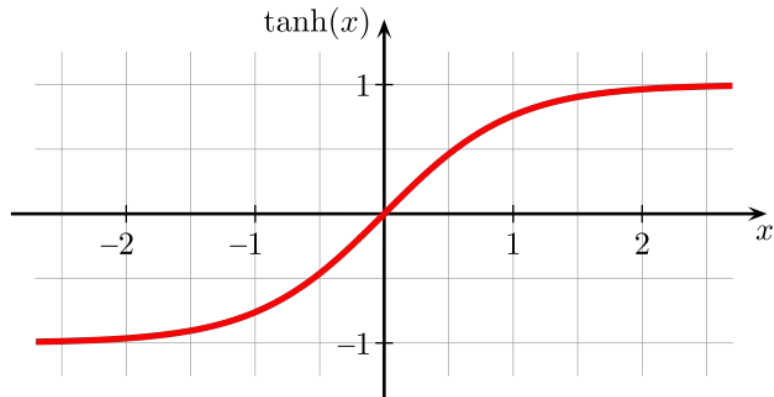
- zamienia wektor na rozkład prawdopodobieństw
- używane na logitach na wyjściu sieci



Tangens hiperboliczny

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

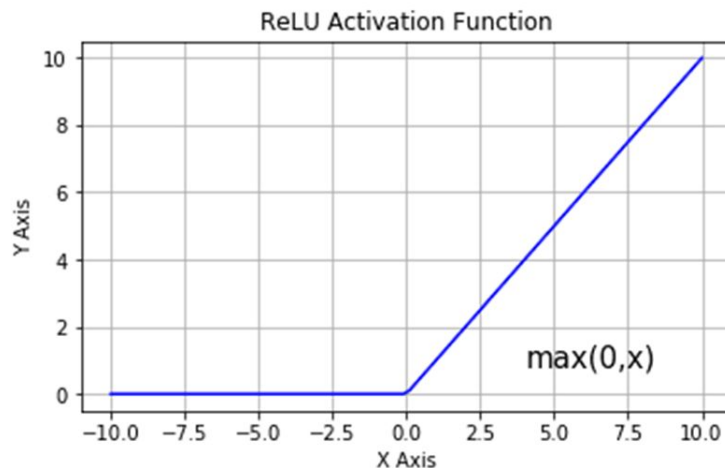
- kiedyś używana, żeby rozwiązać problem nasyconych neuronów z sigmoidy
- dalej ma ten problem
- obecnie nieużywana



Rectified Linear Unit (ReLU)

$$\text{ReLU}(x) = \max(0, x)$$

- przełom w uczeniu głębokich sieci neuronowych
- prosta, szybka, dobrze działa
- nieróżniczkowalna w 0 - robimy if/else
- pochodna w znacznej części to 0, więc neurony się nie uczą, jeżeli wpadną w ten obszar - problem **martwych neuronów (dead neurons)**

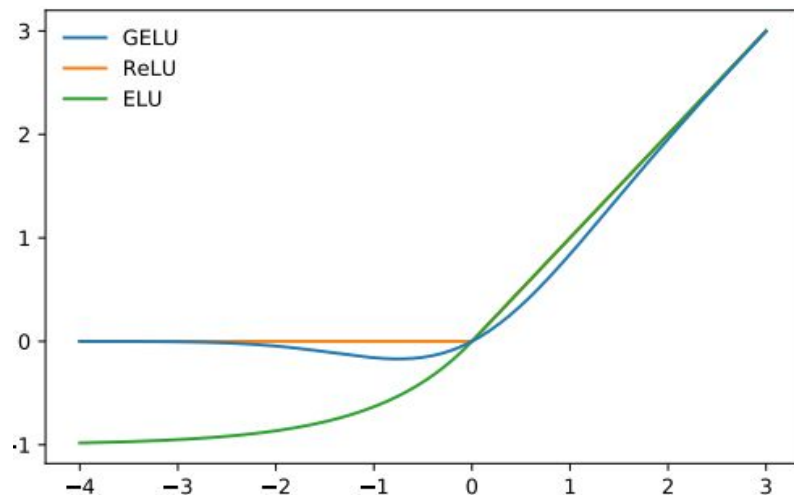


Nowe funkcje aktywacji

- różniczkowalne przybliżenia ReLU
- szczególnie przydatne przy bardzo głębokich i bardzo dużych sieciach
- różne podejścia, koszt i dodatkowe parametry
- ciężko wybrać, popularne są GELU, ELU, SiLU (Swish)

$$\text{GELU}(x) = x \cdot \Phi(x)$$

$$\text{ELU}(x) = \begin{cases} x & \text{if } x > 0 \\ a(e^x - 1) & \text{otherwise} \end{cases}$$



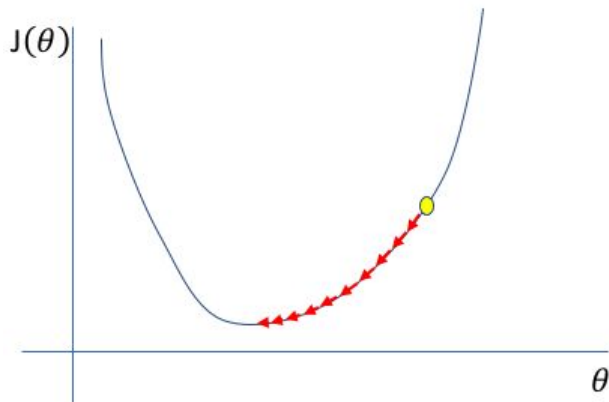
Ulepszenia i hiperparametry

Współczynnik uczenia

- **wyznacza wielkość kroków**
- **duży:**
 - uczymy szybko
 - przeskakujemy minima lokalne
 - ciężko wbić się w minimum
- **mały:**
 - uczymy powoli
 - precyzyjnie znajdujemy minima
 - możemy wpaść w minimum lokalne
- **typowe wartości:**
 - domyślnie: $1e-3$ (0.001)
 - zakres: od $1e-5$ do $1e-2$
- **zwykle stały - stała ucząca**
- **może być adaptacyjny (adaptive):**
 - zależna od momentu treningu
 - różna dla różnych zmiennych
 - dostosowywać się do dynamiki treningu

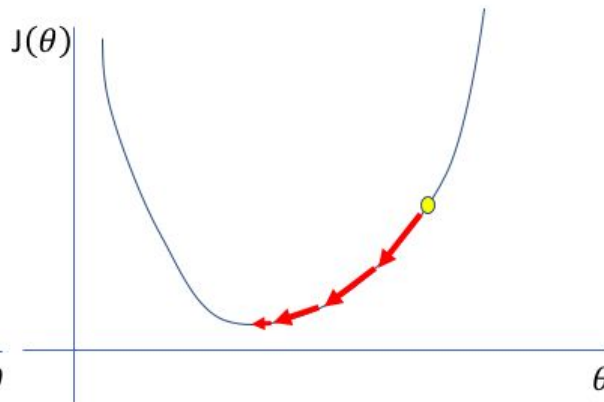
Współczynnik uczenia

Too low



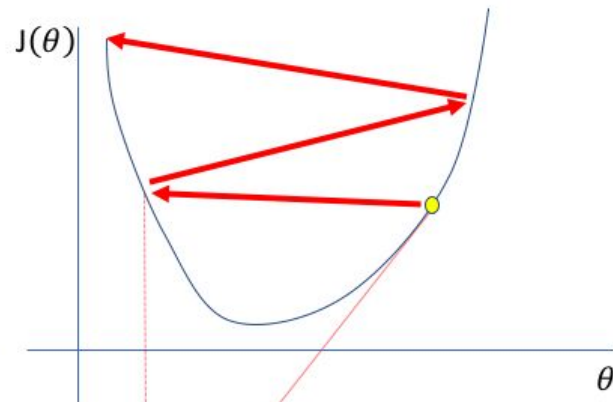
A small learning rate requires many updates before reaching the minimum point

Just right



The optimal learning rate swiftly reaches the minimum point

Too high



Too large of a learning rate causes drastic updates which lead to divergent behaviors

Regularyzacja

- **L2** - tak jak przy regresji liniowej, penalizujemy duże wagi
- **dropout:**
 - losowo wyłączamy neurony w trakcie treningu (zerujemy wagi)
 - zapobiega nadmiernej specjalizacji neuronów - muszą być gotowe, że inne przestaną istnieć
 - często bardzo ostry, np. 25% czy 50%
 - realizuje **ensemble learning** - w każdej iteracji uczymy nieco inną sieć
- **wczesny stop**

Hiperparametry

Architektura sieci:

- liczba warstw
- rozmiary warstw
- funkcja aktywacji

Trening:

- stała ucząca
- siła regularyzacji L2
- prawdopodobieństwo dropoutu
- cierpliwość wczesnego stopu

Pytania?